

Practical: 3

Implementation of max-heap sort algorithm

11/08/2026

Aim: To Implement of max-heap sort algorithm.

Algorithm:

* Algorithms:-
input:- unsorted Array
output:- sorted Array
Step 1:- Start
Step 2:- Build a Max Heap from the given array.
Step 3:- for $i = n/2 - 1$ down to, set
 largest = i
 left = $2 * i + 1$
 right = $2 * i + 2$
Step 4:- If left < n and $A[left] > A[largest]$, set largest = left
Step 5:- If right < n and $A[right] > A[largest]$
 set largest = right
 if (largest != i):
 Swap ($A[i]$ and $A[largest]$)
 Apply the same heapify process to the sub tree
Step 6:- After building the max heap, the largest element is at $A[0]$
Step 7:- for $j = n - 1$ down to 1:
 Swap ($A[0]$, $A[j]$):
Step 8:- Heapify the root $A[0]$, set largest = 0
 left = $2 * 0 + 1$
 right = $2 * 0 + 2$
Step 9:- Compare root with children
 Set largest to the index containing the largest value
Step 10:- repeat step 4 until $j = 1$
Step 11:- The array is now sorted in ascending order
Step 12:- Stop.

Program:

Max-heap sort algorithm

```
#include <iostream>
using namespace std;
```

```
void max_heapify(int arr[], int n, int i)
{
    int largest = i;

    int l = 2 * i + 1;
```

```
int r = 2 * i + 2;

// Check left child
if (l < n && arr[l] > arr[largest])
{
    largest = l;
}

// Check right child
if (r < n && arr[r] > arr[largest])
{
    largest = r;
}

// If largest is not the root
if (largest != i)
{
    int t = arr[i];
    arr[i] = arr[largest];
    arr[largest] = t;

    // Heapify the affected subtree
    max_heapify(arr, n, largest);
}
}

void heapSort(int arr[], int n)
{
    // Build Max Heap
    for (int i = n / 2 - 1; i >= 0; i--)
    {
        max_heapify(arr, n, i);
    }

    // Heap Sort
    for (int j = n - 1; j > 0; j--)
    {
        // Move largest element to the end
        int t = arr[0];
        arr[0] = arr[j];
        arr[j] = t;

        // Heapify remaining heap
        max_heapify(arr, j, 0);
    }
}
```

```
void display(int arr[], int n)
{

    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }

    cout << endl;
}

int main()
{
    int arr[] = {4, 5, 6, 7, 9, 5};
    cout<<"/----- max heap sort -----/"<<endl;
    int size = sizeof(arr) / sizeof(arr[0]);
    cout << "Unsorted array: ";
    display(arr, size);

    heapSort(arr, size);

    cout << "Sorted array: ";
    display(arr, size);

    return 0;
}
```

Output:

```
/----- max heap sort -----/
Unsorted array: 4 5 6 7 9 5
Sorted array: 4 5 5 6 7 9

=== Code Execution Successful ===
```

Time Complexity:

Time Complexity

Left = $2^* + 1 \rightarrow ①$
 Right = $2^* + 2 \dots ②$
 longest = $1 \dots ③$
 height of binary heap = $h = \log n$
 Therefore:
 $(n) : O(\log n)$
 in the heapify = $O(\log n)$

ii) for $i = n/2 - 1$ down to 1
 Max Heapify (n, i)
 there are $n/2$ elements
 $n/2 \times O(\log n)$
 which give $O(n \log n)$

$T(n) = n/4 + 2/8 + n/16 + \dots$
 $T(n) = n(1/4 + 1/8 + 1/16 + \dots)$
 $T(n) = n$
 $T(n) = O(n)$ Build Heapify: $T(n) = O(n)$

new $T(n) = O(n \log n) + O(n)$
 $O(n \log n)$
 $T(n) = O(n \log n)$ $O(n \log n) = O(n)$
 $n \log n = h$
 Best case: $O(n \log n)$
 Average case: $O(n \log n)$
 worst case: $O(n \log n)$

Conclusion:

Heap Sort was successfully implemented using a Max Heap. It efficiently sorts the elements in $O(n \log n)$ time and requires $O(1)$ extra space.